MoE Megakernel: Internode Extension

Two-level Expert Parallelism: NVLink intranode + NCCL GIN over IBGDA

Minimax M3 Pro · B300 · CX8

built on the intranode Megakernel; internode layers on top

Background: from one NVLink domain to internode EP

Where we start

- MegaMoE: one persistent megakernel per MoE layer
- Expert parallelism (EP) inside one **NVLink domain**

What this work adds

- A second EP level across nodes, so one layer spans **world = 32 (4 nodes × 8), 384 experts, 12 per rank**.
- Device-initiated RDMA through **NCCL GIN over IBGDA**: `gin.put` from the kernel, no host, no copy engine.

What the deck shows

- Design (July): why internode RDMA is a communication floor, what overlap can and cannot buy.
- Implementation (August): four stages shipped, a world=32 memory-corruption bug traced to one zeroing stride.
- Tuning against real baselines (September): production N4R8 **3138 → 495 µs, 0.86× NCCL-EP, 0.76× DeepEP v1**.

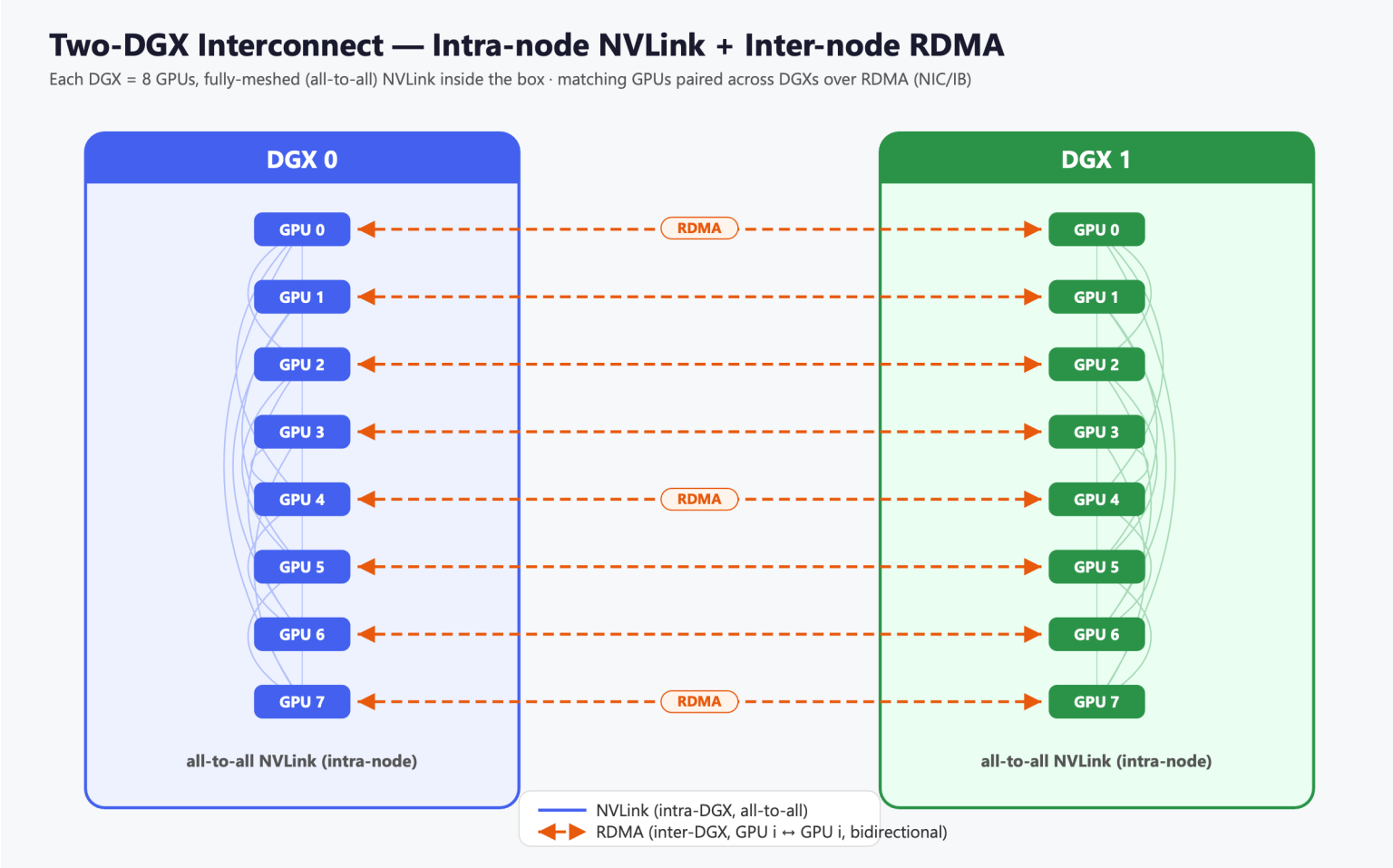
Scope: a two-level EP hierarchy

Level 1: Intranode (unchanged)

Level 2: Internode (new)

- EP across nodes via **NCCL GIN over IBGDA**
- each rank talks RDMA only to its **relay peer** (same `local_rank`) on each remote node
    - QP count per GPU *O(N)*, not *O(NR)*

The GEMM warps (TMA-B, MMA, epilogue) are **agnostic to token origin**.



Notation: `N` nodes, `R` ranks/node, `E` experts/rank, `T` tokens/rank, `world_size = N×R`.

Can the MoE layer hide internode RDMA?

Not by compute: ~3× internode roofline gap
Nor fully by weight load: ~55 µs, vs ~125 µs bf16 combine

So: the combine RDMA is exposed; the layer's overlap hiding is **limited**.

Roofline: hide ratio vs model intensity

Model intensity = FWD FLOP ÷ comm-bytes = (6hd) / ((b_d + b_c) h)

wire (disp + comb)	comm	intensity	qwen3	DSv4 Pro
fp8 + bf16	*3h*	*2d*	2,048	6,144
fp4 + fp8	*1.5h*	*4d*	4,096	12,288

Hardware hide ratio: TFLOPS / TB·s

GPU · link	ratio	GPU · link	ratio
GB200 · NVLink5	~5,500	H200 · CX8	**~20,000**
H200 · NVLink	~4,444	H200 · CX8+DeepEP	~10,000

- **Intranode (NVLink):** hide ratio 4.4–5.5k ≈ model 2–6k → overlap pays. This is what the intranode megakernel already does.
- **Internode (CX8):** hide ratio 20k is **~3× the model intensity** → the MoE GEMM's **compute** cannot completely hide it. Nor does weight load rescue it: see next slide.

Consequences

1. Overlap can't completely hide comm: combine (IB) is the bottleneck, and FC, NVLink hides under *it*, not the reverse.
2. Every byte matters → two-stage relay + node dedup

Model intensity. FWD FLOP = FC1 *4hd* + FC2 *2hd* = *6hd*; comm = dispatch *b_d h* + combine *b_c h* (bytes/elem: fp8 1, bf16 2, fp4 ½).

Internode comm = *2d × (k) / (E[distinct nodes])*.
Under balanced routing, *E = N(1-(1-1/N)^k).* For *N=4, E = 1.5.*

Time budget: per-stage estimates (decode)

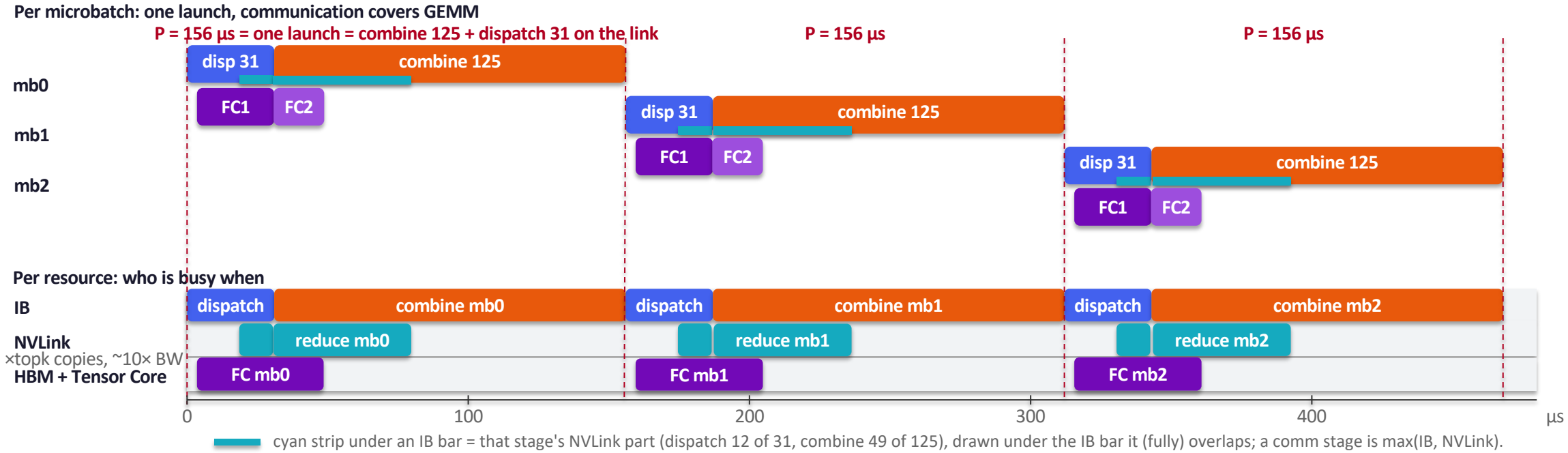
Hardware: B300 · peak → achievable

resource	peak	×factor	achievable
FP4 dense compute	15 PFLOP/s	0.67	~10 PFLOP/s
HBM3e	8 TB/s	0.75	~6 TB/s
NVLink5 (1-dir)	900 GB/s	0.67	~600 GB/s
CX8 IB / rail	100 GB/s	0.60	~60 GB/s

Stage	decode T=256	note
Dispatch	~31 µs	IB out, node-deduped (~35 µs / ~160 µs End at the full 4056B message)
FC1	~37 µs	weight-bound (compute ~8)
FC2	~18 µs	weight-bound (compute ~3)
Combine (bf16)	~125 µs	**IB back: the bottleneck**
End	**~156 µs**	max(FC1,Comb)+max(FC2,Disp)
bottleneck	*IB (combine)*	FC (~55) hides under it
Serial	~211 µs	Disp + FC1 + FC2 + comb

Model: H=7168, fused gate+up J=5120 (post-SwiGLU I=2560, the production runner shape), topk=4, **12 experts/rank** (384 total; world 32 = **4 nodes × 8**), NVFP4, **bf16 combine**; weights ≈ 0.33 GB/rank (FC1 220 MB, FC2 110 MB). Comm/rank crosses IB to ~2.05 distinct remote nodes (N=4, k=4, dedup); FC = max(compute, weight load); End = max over shared-resource totals.

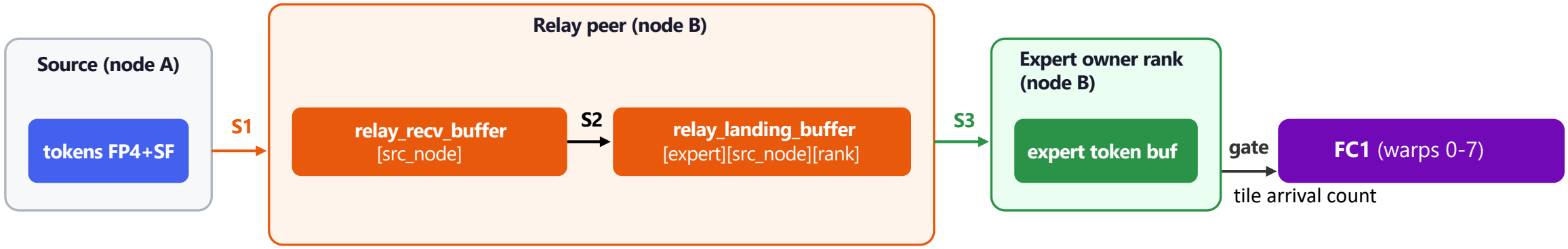
Microbatch overlap: can't hide a comm bottleneck



- **IB is the saturated lane**; NVLink and FC (weight-bound) both sit in its shadow

Design and Implementation

3-stage dispatch



Stage 1: RDMA

- put token → node's *relay peer*
- token deduplicated: once per destination node, regardless of experts hit
- saves bytes by up to O(N) times

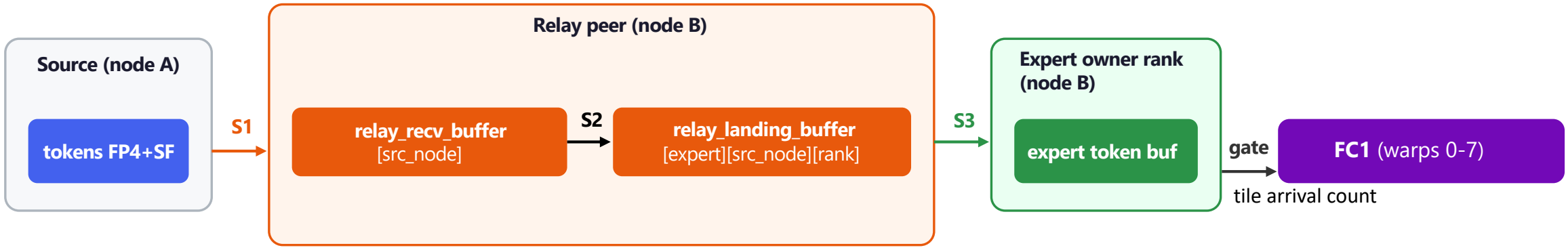
Stage 2: NVLink fan-out

- *relay peer* forwards each token → the *expert ranks* it targets (intranode)
- This relayed design avoids cross-rail connection, and saves QP by *relay_peer* times

Stage 3: relay copy

- *expert owner* repacks the landed tokens into expert's token pool (intra-rank)
- Expert token pool becomes single-writer
- These 3 sub-stages can overlap each other

3-stage dispatch (FP4)



Stage 1: RDMA

Stage 2: NVLink fan-out

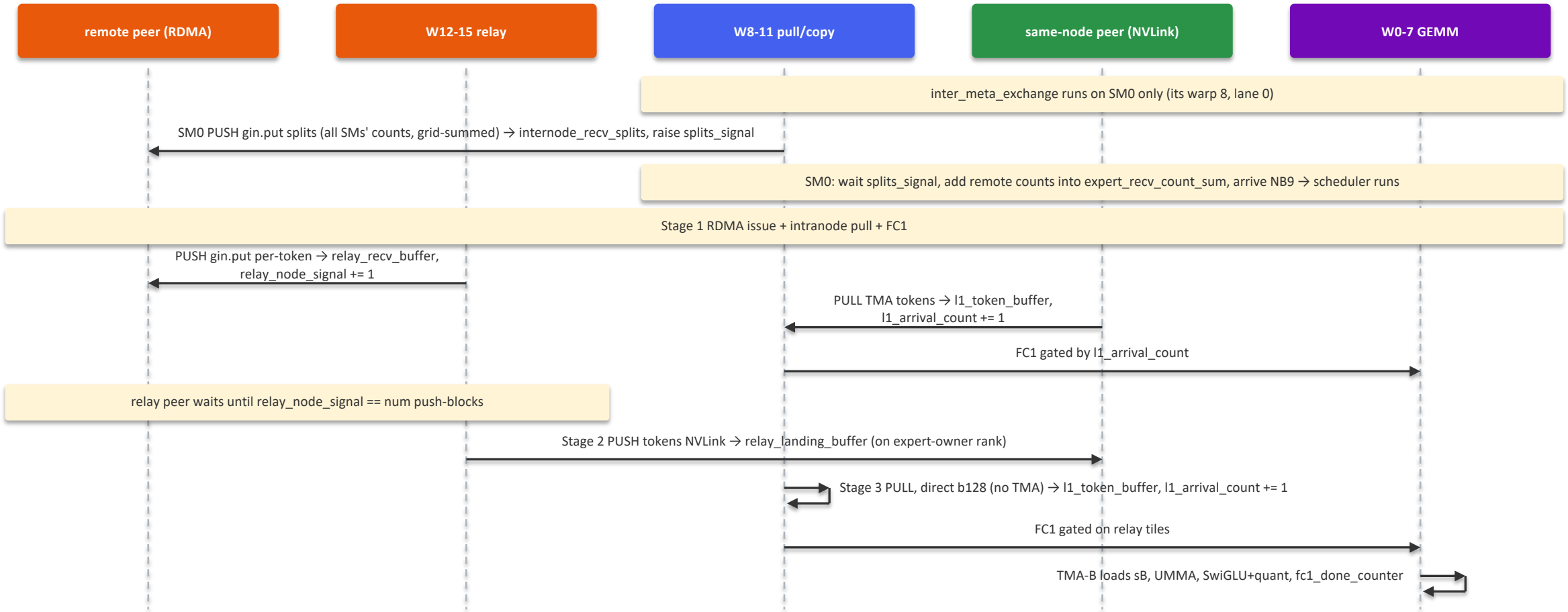
Stage 3: relay copy

combine (BF16): symmetric, 2 stages

- **Stage 1** NVLink gather-reduce onto relay peer, fp32 accumulate
- **Stage 2** RDMA back remote tokens

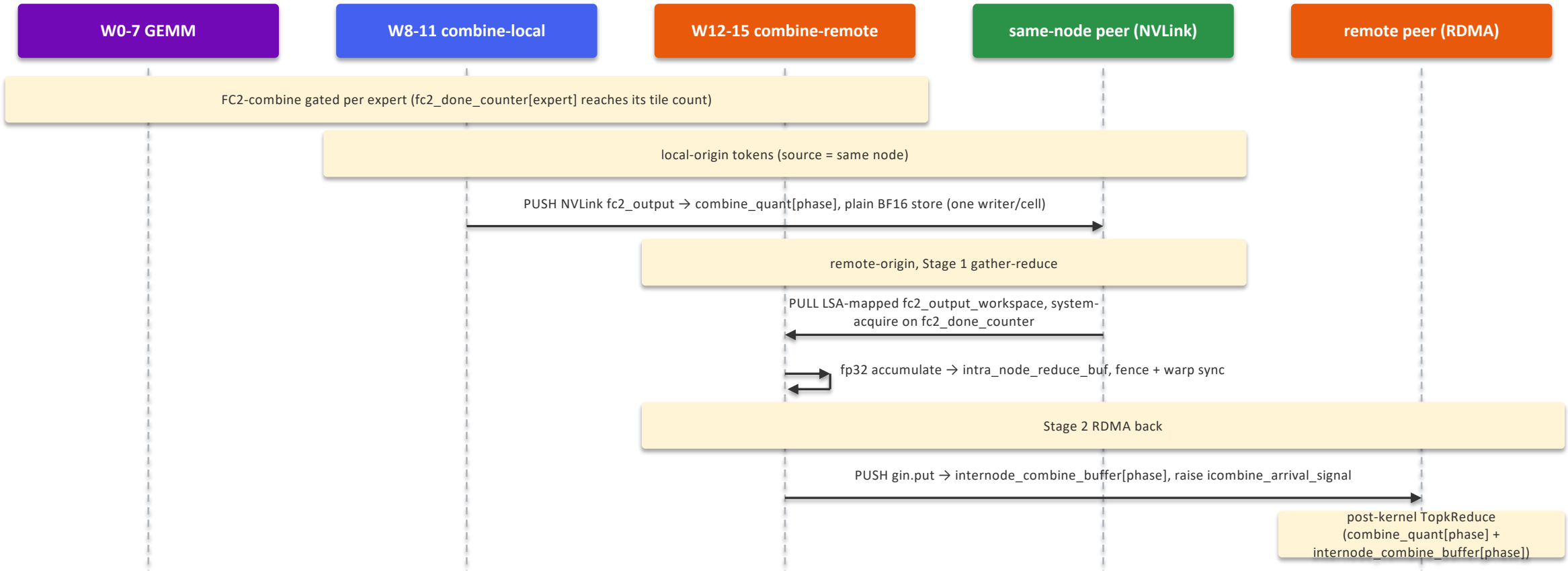
- Post-kernel **TopkReduce** sums up both remote and local tokens across topk.

Dispatch: who moves data, how it syncs, where it lands



One rank is both **relay peer** (receives RDMA, fans out) and **expert owner** (relay_copy into pool). **PUSH** = gin.put (RDMA) or NVLink store · **PULL** = TMA copy or peer read · **NB9** = NamedBarrier(9), a CTA-wide barrier · **+= 1 / raise** = increment a counter / signal.

Combine: who moves data, how it syncs, where it lands



Reduce is a **PULL** (gather many→one in one fp32 register, avoids multi-writer contention). Stage 2 **PUSH** back, one per-node signal. Topk weight pre-multiplied before FC1 (bf16 path). `phase = internode_barrier_counter & 1` double-buffers both planes across launches.

Communication substrate: NCCL GIN over IBGDA

gin.put: one SM thread posts a descriptor, the NIC does the transfer

ncclCoopThread() scope: no warp cooperation, no GPU copy engine on this path



- Warp-scope put = 32 *independent* WQEs in a batch (opposite of NVSHMEM's warp put): only helps for many small puts to different peers.

Object-style API, **window + offset** addressing:

```
gin.put(coop, team, peer,
        dstWin, dstOff, srcWin, srcOff,
        bytes, ncclGin_WeakSignalInc{idx})
gin.waitSignal(coop, idx, expected)
gin.flush(coop)
```

- **Not a copy engine**: only the NIC's DMA engine moves the data, unlike a host-launched `ce_coll.cc` collective.

- Signals are index counters, no raw SET/ADD. The plain `SignalInc`/`SignalAdd` are deprecated; pick **`Weak`** (rides the very put it certifies) or **`Strong`** (certifies a batch of earlier puts) deliberately.

From design to production

Four implementation stages shipped,
and a three-hop memory-corruption bug at world32/R8 traced through two full core dumps

Implementation staging

Step	Deliverable	Verify
0	Intranode path with 16-warp layout, W12-15 idle	NB(10)=256 / NB(9)=288 / NB(8)=512 don't break existing flow
1	`inter_meta_exchange` + pool layout	`relay_pool_base`, per-source counts vs CPU reference
2	Stage 1 RDMA + Stage 2 fan-out	tokens land in `relay_landing_buffer` with correct routing
3	relay_copy + FC1/FC2 over relay tokens	FC2 outputs match single-node reference
4	two-stage combine + TopkReduce	full internode output vs all-on-one-node / NCCL-AllGather

Intranode-first: the extension does not touch the FC1/FC2/epilogue core (warps 0-7), the AB pipeline, or the ACC pipeline.

The production-scale crash that cost a 70GB core dump

Problem. world=32, R=8, the target topology, faulted with a Warp MMU Fault on its third launch, inside the RDMA push.

Finding the root cause. Two full 71.8 GB CUDA core dumps decoded with cuda-gdb, walking the chain backwards from the fault.



Cause, three hops upstream. A barrier was widened from 128 to 256 threads in `inter_meta_exchange` and the loop zeroing each CTA's shared per-expert counter followed that stride by mistake: experts 128 to 255 were never cleared.
→ Stale counts let the pool layout overran the L1 buffer, `dispatch_pull` overwrote the neighbouring token table, → `internode_push` wrote to garbage idx.

Correct: stride = num_dispatch_threads (128)



Bug: stride = dispatch_group_threads (256)



Lesson: the test matrix missed the one cross term that mattered.

The bug needs three things at once: the relay-enabled 128-thread writer, more than 128 total experts, and a second launch (shared memory keeps the previous round's residue). Our tests failed to cover them all at once:

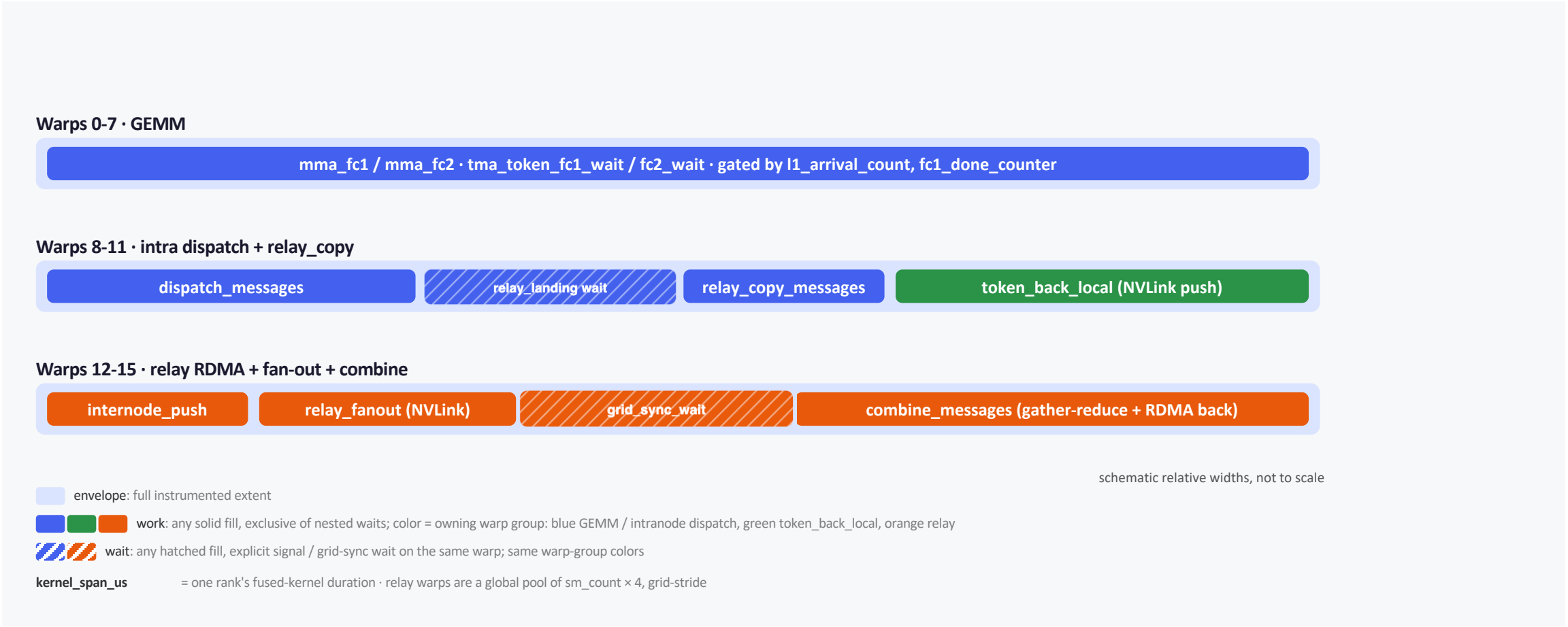
run	repeated launch	E > 128
world=4, R=2	yes, 1 warmup + 3	no, E=8
world=4, 12/rank	yes, 5 + 20, bitwise exact	no, E=48
later world16 large topology	no, or no per-launch check	yes, E=192

"Repeated launch × E > 128" was never run.

Tuning against real baselines

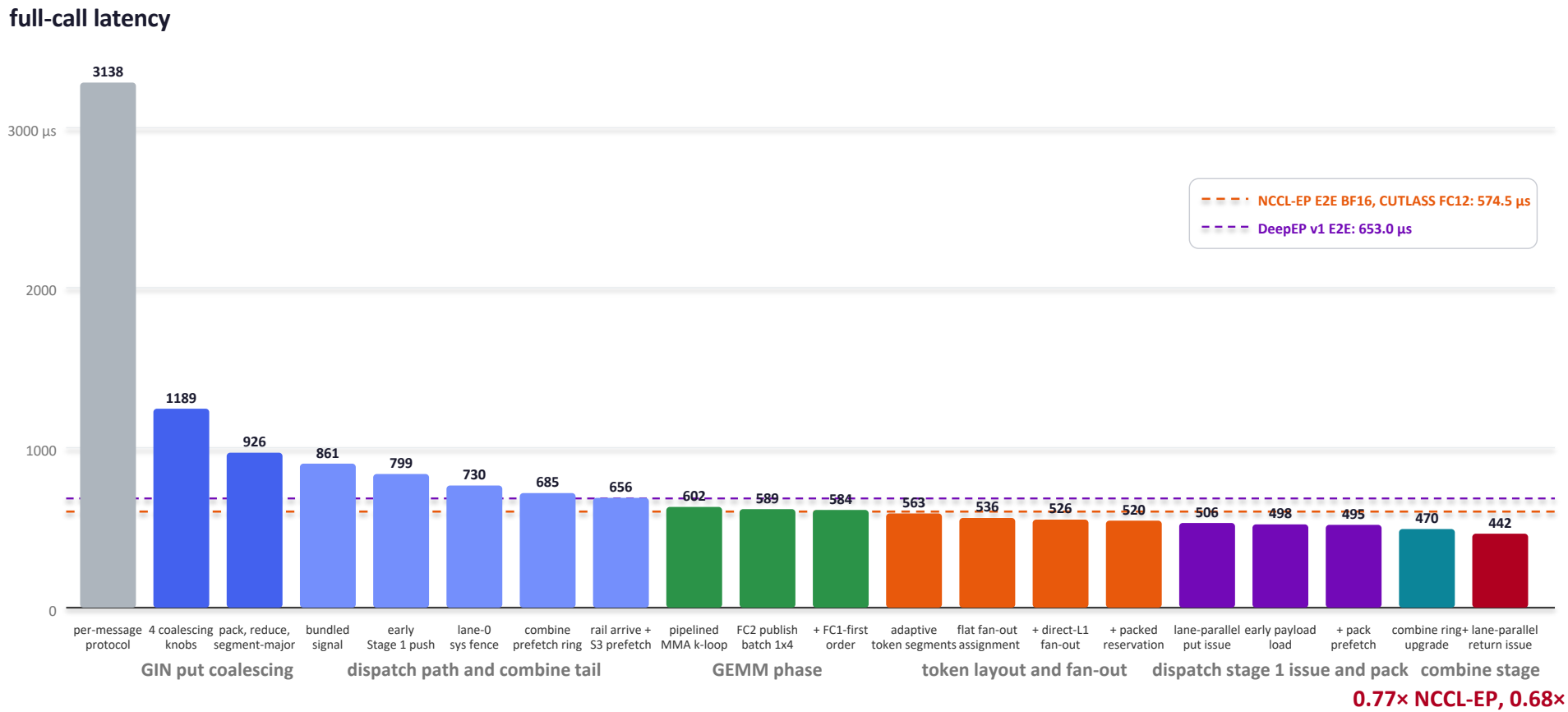
Gated in one allocation against a pinned NCCL-EP end-to-end baseline (CUTLASS FC12, BF16 combine) and a DeepEP v1 NVFP4 end-to-end run.

Measuring it first: IKET + PIC-C profiling instrumentation



`kernel_span_us` per rank, plus per-warp `megamoe_envelope` / `_work` / `_wait` ranges
structural checks (`BLOCK_CONCENTRATION`, `WAIT_WORK_IMBALANCE`, `SPARSE_WORKFORCE`) flag an imbalanced work partition before anyone has to eyeball a trace.

Results at Production N4R8, T=256: 3138 → 442 µs, 0.77× baseline



0.77× NCCL-EP, 0.68× DeepEP

Statistic is per-iteration rank maximum CUDA event, averaged over 20 timed iterations (`max_rank_mean_us`), the same figure the NCCL-EP tester reports and the latency the caller sees. Same-allocation ABBA controls launch skew. Every step passed the 32-rank repeated-launch exact correctness gate.

1. RDMA granularity: per-token puts, initially



- internal `libgpunetio` result: 512 B already saturated the link
- Our relay_msg_bytes ≈ 3.5KB >> 512B; demanded rate ≈ 17 Mmps, roughly 10× under the NIC message-rate ceiling
- Dropping the range coalescence removes a step from the critical path.

- **Overturned during implementation:** the per-put SM issue cost was completely overlooked in this design. Each put costs ~1.4 µs to write its WQE entry on the GDAKI context regardless of bytes.

- Lesson: Reproduce before adopting existent performance numbers

"per-token puts are free" did not survive contact

Before: one post per relay message, one per returned partial

msg msg msg msg msg msg msg msg

×240 messages per rank at N2R2: about 240 posts on one queue pair, each CAS + lock + doorbell

cost scales with puts: a 14336B row needs ~0.3 µs at 50 GB/s

After: pack in place, one range put + bundled count per destination node

relay_send_buffer[node][0:count) is already dense: **one gin.put per dest node**

SM0 issues 1 put with a bundled `StrongSignalAdd`(expert hits) per node after a relay-only grid sync. Combine returns intra_node_reduce_buf[src][slot] the same way; TopkReduce reads via token_node_slot

publish_issue 399 → 19 µs at N2R2

Four coalescence knobs: warp-pack, dispatch coalesce, combine coalesce, parallel Step 5b redistribution

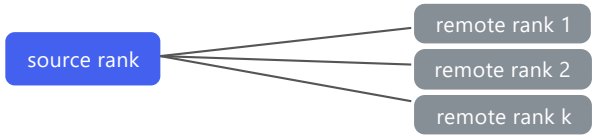
N2R2 T=256: 1573.6 → 655.7 µs (-58.2%) · N4R8 T=256: 3138.3 → 1189.2 µs (-62.1%, orders within 0.1%)

Token sweep (N2R2, job 2924733): -31.6% at T=8, -58.2% at T=256, -90.3% at T=4096

- The initial design had treated per-message puts as free and only counted bytes. The bytes were never the dominating cost.

Related works: why the low-latency kernels send one put per token

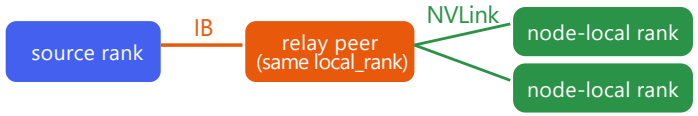
A. Per-token point-to-point (DeepEP LL, NCCL-EP LL, DeepEP V2 direct)



- one put per (token, destination): DeepEP per top-k expert, NCCL-EP per destination rank;
- No SM residency: slots by local atomicAdd, counts by trailing AMO; no handshake, issue and exit

Production, per rank, T=256: 765 / 731 puts + 288 signals; combine 765 puts of 14.5 KB

B. Relay + coalescence (DeepEP normal, NCCL-EP HT, DeepEP V2 hybrid, MegaMoE)



- DeepEP normal: 6-token chunks; NCCL-EP HT: 64-token chunks, one doorbell;
- SM-resident forwarding

MegaMoE: one range put per (topk-1) remote nodes: 3 puts, 525 messages, signal bundled; combine 3 puts

They prioritize send-issue latency and 0-SM residency, rather than per-message cost.
DeepEP #179: "issue lots of instructions and just exit the kernel … the real data sending does not occupy any SM"; #95: NVLink forwarding "conflicts with hook-based overlapping".
NCCL-EP "adopts DeepEP design". Both coalesce only in their high-throughput modes, MegaMoE's two-hop topology.

What it means here. The gain scales with tokens per destination: 30× at EP=32, 4× per rank at EP=256 (32× per node, with SM-resident forwarding), which is why LL skips it at DeepSeek's scale. Skew hurts the per-token scheme, not coalescing.

Coalescing pays by tokens per destination (T·K / N_dest), not by how balanced the routing is

configuration	per-token puts + signals	per-rank coalesced	per-node coalesced (messages)	hot receiver, messages, median / max	per-token ingress MB, median / max
N4R8 E12 T256 K4, uniform (production)	734 + 270	24	3 (528)	732 / 794	3.00 / 3.25
same, Zipf 1.0	776 + 184	24	3 (505)	512 / 3623	2.10 / 14.84
same, 5% hot experts take 50%	726 + 220	24	3 (517)	683 / 1468	2.80 / 6.01
EP64 N8R8 E4 T128 K8, uniform	860 + 220	56	7 (592)	860 / 909	3.52 / 3.72
EP256 N32R8 E1 T128 K8, uniform	992 + 244	244	31 (902)	993 / 1107	4.07 / 4.53
same, Zipf 1.0	1002 + 195	195	31 (902)	381 / 25025	1.56 / 102.5
EP256 N32R8 E1 T32 K8, uniform	248 + 158	158	31 (225)	248 / 299	1.01 / 1.22

Skew multiplies the hot receiver's per-token messages and bytes about 7×;
node coalescing plus relay-peer routing spreads that node's RDMA ingress over its 8 NICs (max 2.97 MB against 2.15 uniform).
EP=32: per-rank coalescing already saves 30×; EP=256 T=128: 4× per rank, 32× per node.

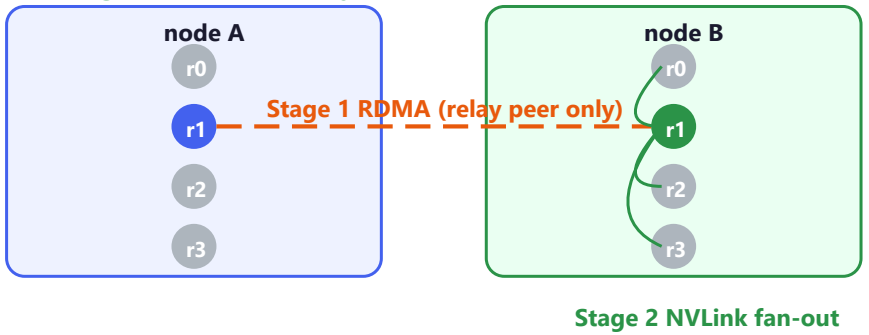
Sources checked on 2026-09-14: DeepEP 58397ea / v1.2.1 / main 01dc3aa, nccl-extensions 6a81bce, DeepSeek-V3 and NCCL-EP papers, maintainer issues, internal audits. Model: scripts/coalescing_routing_model.py, 4096 B per message.

Related works: where we sit relative to DeepEP V1/V2

Two ways to cross nodes: rail-restricted relay vs full-mesh direct

Same NCCL Gin substrate as DeepEP V2; the difference is topology and usage, 4 ranks/node shown for clarity

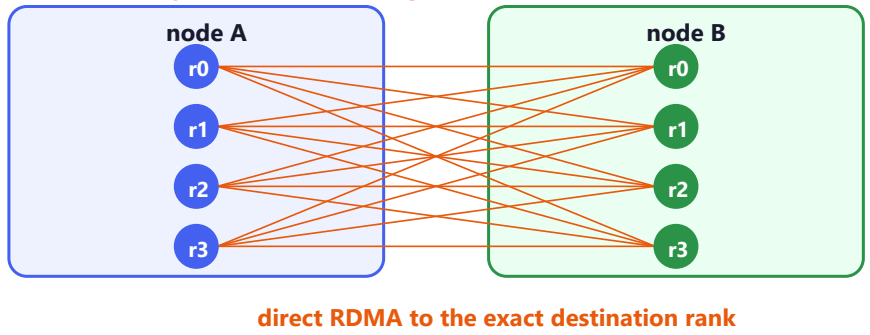
This design · DeepEP V2 hybrid (two hops)



QP / GPU: O(N)

rail-restricted: only same-local_rank peers connect over RDMA

V1 low-latency / V2 direct (single hop)



QP / GPU: O(world)

lowest latency, needs full-bisection fabric; DeepEP does not default to this across nodes

Overlap model: where communication and GEMM actually share a clock

This design: tile-granular, one megakernel

warps 0-7 (GEMM) and warps 8-15 (comm) share one CTA;
a landed 128-token tile releases l1_arrival_count and FC1
consumes it immediately, intranode or relay alike

overlap inside one microbatch

DeepEP: comm kernel

4-6 SM, IBGDA

overlap only across microbatches

PDL / hook

DeepEP: GEMM kernel

standalone grouped GEMM

Both generations share the NCCL Gin substrate. This design is structurally DeepEP's **V2 hybrid** path, the same one DeepEP defaults to across nodes (`allow_hybrid_mode=True`). The difference is synchronization style: this design is signal-gated streaming for a fused megakernel; V2 is barrier-centric around a standalone kernel.

2. Fan-out publish: 608 warps × 32 system fences serialize; one lane-0 fence does not

fan-out envelope split: at N2R2, route decode 2.8 µs, row copies 1.7 µs, publish 52 µs

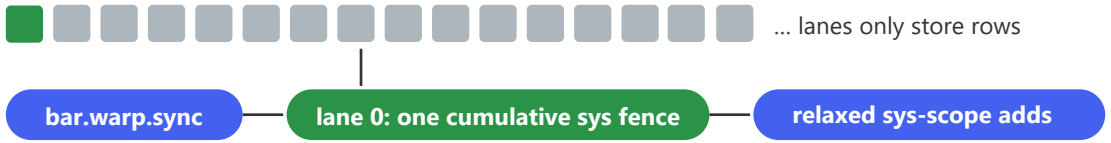
Before: every lane fences after its stores



then one release add per expert hit

N4R8 publish 120 µs

After: MEGA_GIN_LANE0_SYS_FENCE=1



same bytes, counters, and orderings; fewer fences

N4R8 publish 120 → 38 µs, -8.3%

Applied at the fan-out stage (both dispatch and combine paths).

3. FC2 tiles are epilogue-bound; one system-scope release per tile throttles the MMA warp

1×1: one release per FC2 tile



8.3 µs per tile, epilogue-bound
epilogue span 8.3 (p50); the MMA warp shows the stall as mainloop 3.3 → 6.8

1×4: one release per four FC2 tiles



3.3 µs per tile, MMA-bound
epilogue p50 1.6; four lanes fire four red.release.sys at once, one fence.

FC2 epilogue p50 8.3 → 1.6 µs · epilogue per CTA 76-82 → 31-33 µs · FC window 125-129 → 104-107 µs at N2R2; production window 187-191 → 162-164 µs

Cost of batching: an expert's last publish can wait (batching-1) more tiles. 1×8 is but 0.8% slower at production (combine FC2 waits +5 µs, FC stage -4 µs).

Per-tile FC timeline, rank 0, N2R2, consecutive FC2 tiles.
Each tile's fc2_done_counter increment is a red.release.sys (SysReleaseFlagBatchTracker), fired after the 128-thread barrier and the epilogue-end stamp of tile k, so its stall opens tile k+1's span.
Attributed by A/B, not by direct measurement of one fire: the release stall length per tile and its share of the MMA slowdown need a SASS inspection or a per-warp stamp around the fire and a membar-stall counter pass.

4. Layout: token segmentation can cost a whole tile

- Initially: intranode and internode tokens are staged into two respective segments per expert, for better overlapping
- Cost: one more tile (one more time FC weight load) under the customer's T=256 config -- 77 to 97 routed tokens per local expert, of which 18 to 25 intranode and 55 to 73 relay

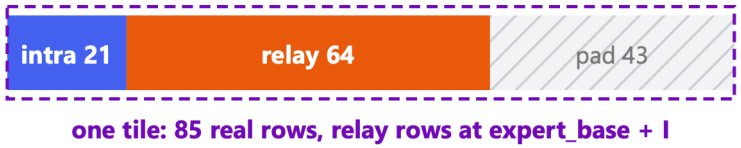
Solution: Adaptive token segments; one 128-row tile per expert instead of two

Before: two segments, each padded to a tile



weights streamed once per tile
per rank: 960 FC1 and 1344 FC2 tiles, 1.32 GB of weight traffic per layer
intranode tile can start before the relay rows land (that overlap is what merging gives up)

After: MEGA_MERGE_TOKEN_SEGMENTS=1, merge when it saves a tile



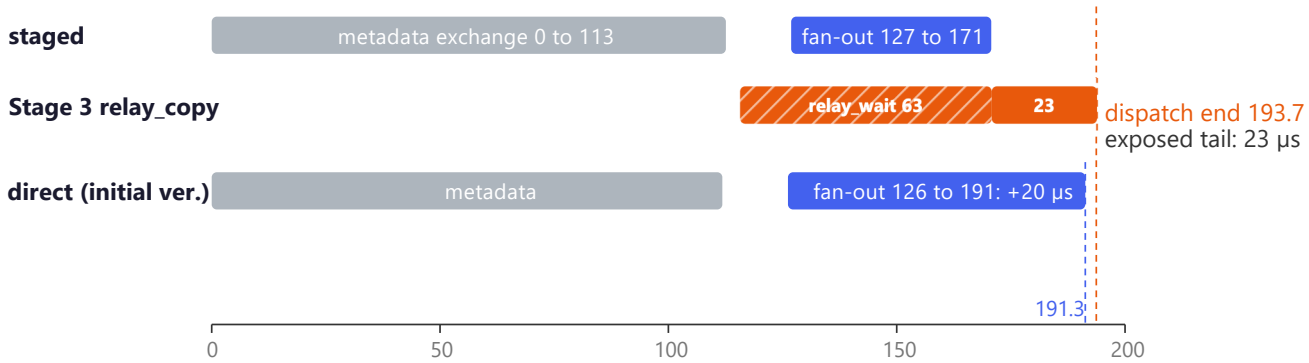
per rank: 480 FC1 and 672 FC2 tiles, 0.66 GB of weight traffic
Adaptive merge, iff $\lceil (I+R)/C \rceil < \lceil I/C \rceil + \lceil R/C \rceil$, per expert (true at T=256, topk 4; false at topk 8 or T=512)

Production config: 585.186 → 562.962 µs (−22.2 µs, −3.80%)

saves exactly one tile per expert, only when both partial tiles fit in one; the gain shrinks with tiles per expert, but never goes negative; all numbers are on near-uniform routing (77 to 97 rows per expert).

5. Relay stage coalescence?

Tradeoff: Maintainability-- Performance++



Every direct-L1 gate, production pair means

- direct v1: all-lane fence, per-hit release
  +15.3%
- direct + lane-0 cumulative fence
  +2.4%
- direct + 4-lane concurrent reservation
  −0.36% (N2R2 +0.58%)
- flat fan-out assignment alone
  559.0 → 535.6, −4.19%
- flat + direct
  562.1 → 526.1, −6.42%
- flat + direct + packed reservation
  528.5 → 520.0, −1.60% (N2R2 −0.88%)

direct-L1 gains: the 23 µs Stage 3 tail, the dispatch warps freed during the FC window (N2R2 epilogue per CTA 34.8 → 22.7 µs, less memory-pipe contention on the SM)**, and a shorter combine tail.**
What it costs per message: a remote system-scope atomic reservation (about 11 µs at N2R2 even after packing the three acquire loads into one Int64), and the per-hit SF scatter, 112 scalar swizzled remote stores per hit (fanout_copy about 14 µs per message). With three messages on the busiest warp those costs ate the whole gain; with the flat assignment they are paid once and −36 µs appears (3050278).

Lifecycles overlap; only the critical path adds up.
Left: production dispatch path, staged vs direct arms of job 2992615, 32-rank means of the last timed launch, µs after each rank’s dispatch start. Right: every direct-L1 ABBA in order, production pair means.

6. the fused GEMM ran 5× slower per k-slab than the same kernel alone

GEMM phase: the MMA warp's issue chain sets the slab rate

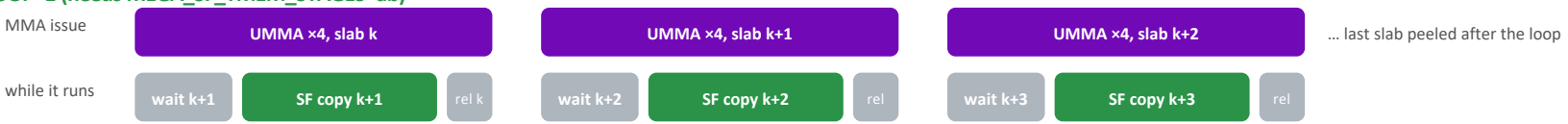
N2R2 FC window 149 to 155 µs vs 66 µs for the same fused kernel without communication warps. One 36 KB k-slab: ~1280 SM cycles in the megakernel vs ~250 with L2-hot weights on the lean kernel.
TMA-B waits 89% for a free stage, MMA waits 4% for data, tensor pipe busy only 46% (89% lean), every TMEM and operand interlock counter zero. ILP bound!

Before: each slab's chain in program order



The compiler's unroll hint (MEGA_MMA_KLOOP_UNROLL=2/4) keeps this order and is 15% / 11% slower on the lean kernel.

After: MEGA_MMA_PIPELINED_KLOOP=1 (needs MEGA_SF_TMEM_STAGES=ab)



The prefetched copy needs a TMEM slot the in-flight UMMAs do not read, hence one scale-factor slot per AB stage; the full-barrier wait already orders it, no new sync.

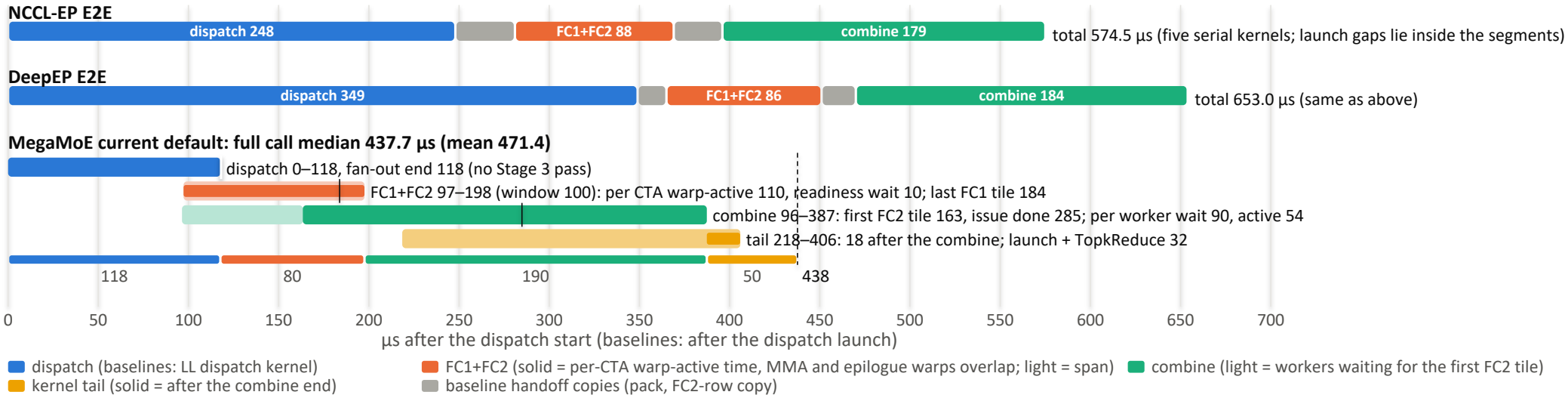
FC1 tile mainloop 21.3 → 14.5-15.5 µs · FC2 tile 7.0 → 3.3-3.5 µs · FC window 149-155 → 124-127 µs (job 2959143) · exact at both scales, 5 of 5 relaunches

Same-allocation ABBA: N2R2 412.7 / 411.1 → 390.6 / 389.2 µs (-5.3%, job 2959463); N4R8 655.5 / 658.2 → 601.7 / 603.0 µs (-8.3%, job 2959465); 1.14× → 1.05× NCCL-EP. Default since f8fbb58 (suffix _kpipe).

The MMA warp now waits for data 49 to 66% of its loop: the five-stage AB ring is operand-bound, not issue-bound, so shortening the chain further is no longer the lever.

Side by side with NCCL-EP and DeepEP at production

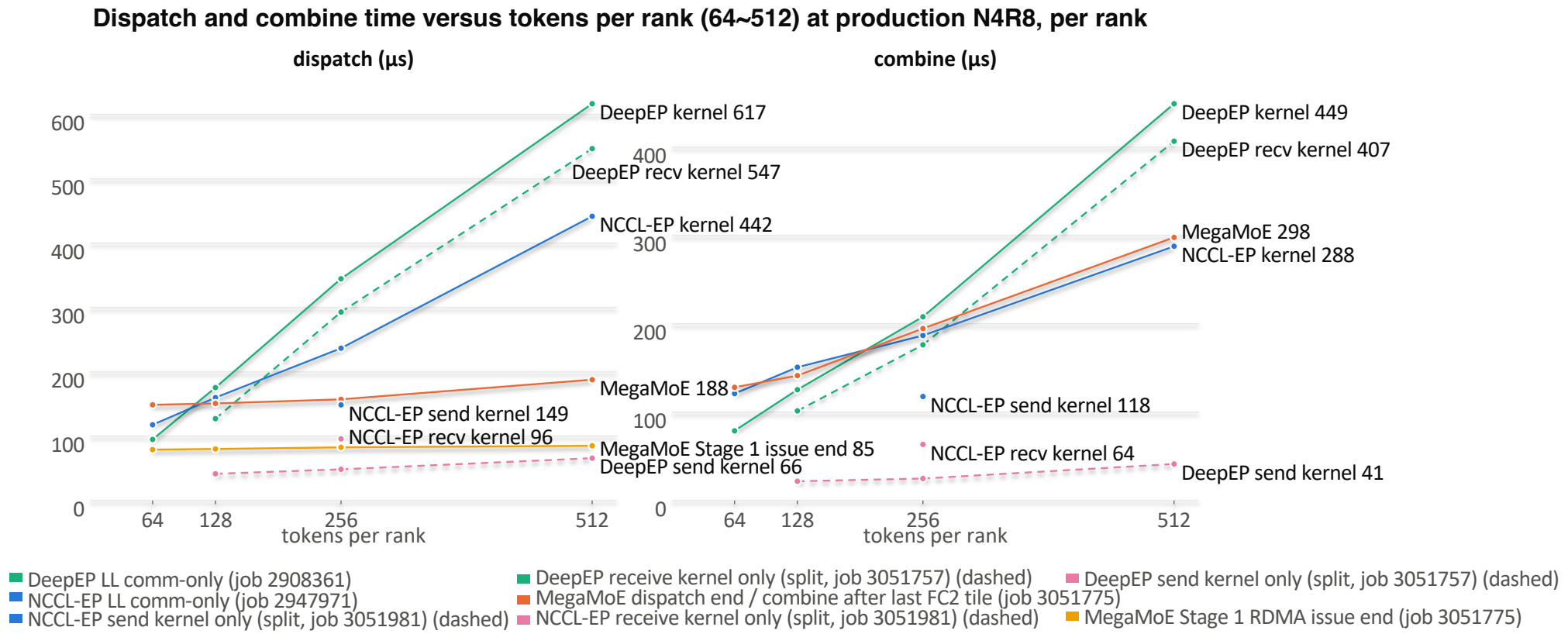
Stage spans at production N4R8 (T=256, J=5120, E=384) one timed run per arm, medians over 32 ranks



Spans are stage lifecycles (first real action to last); they overlap, so their lengths do not sum to the call time. The thin bar under each MegaMoE block is the critical-path partition, which does sum to it.

MegaMoE is now ahead in every shared slice and pays only the 50 µs after the combine that the baselines fold into their kernel deltas.

Side by side with NCCL-EP and DeepEP at production



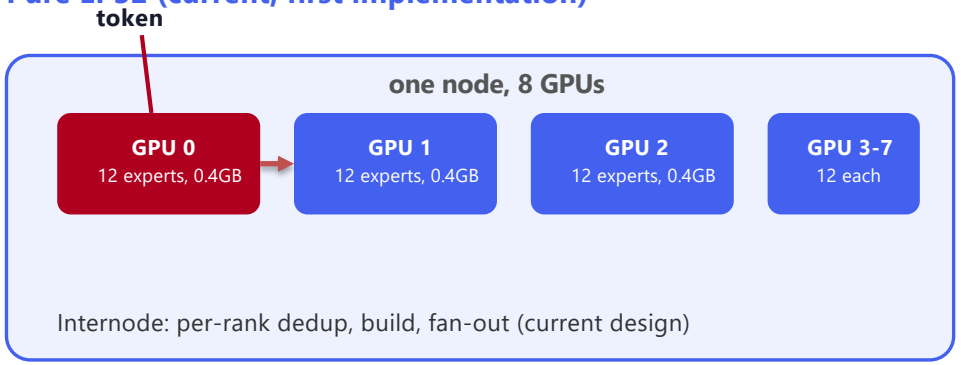
Baselines: kernel time of the stage (dashed: send-only and receive-only kernels of the split runs). Medians over ranks; lines join measured points only
MegaMoE: dispatch end and Stage 1 RDMA issue end; combine from the last FC2 epilogue to the combine end.

A later-phase alternative: EP4×TP8

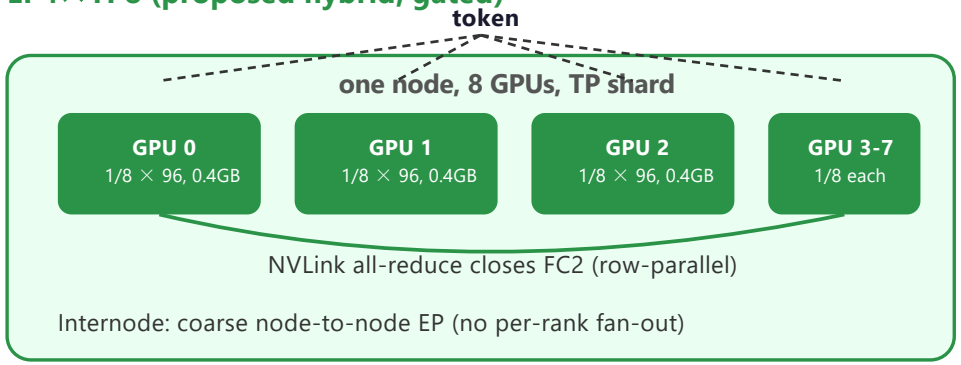
Same hardware, two ways to split it: EP32 vs EP4×TP8

Per-GPU weight bytes and GPU count are identical; only the logical unit shifts from "12 whole experts" to a 1/8 TP shard of a node's 96

Pure EP32 (current, first implementation)



EP4×TP8 (proposed hybrid, gated)



The gate before building it

FC1 is column-parallel (no communication); FC2 is row-parallel and closes with an intranode NVLink all-reduce, the same work TP always needs. Internode becomes coarse node-level EP: one RDMA transfer per destination node, no per-rank relay build.

What it does NOT change: the cross-IB byte count.

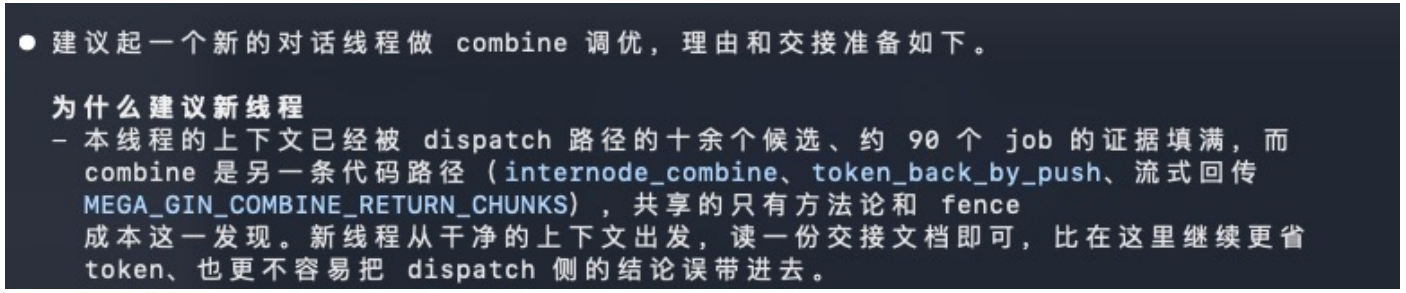
Prototype this only if the relay's fixed latency is a material fraction of the dispatch step.

Summary

- Internode RDMA **cannot be fully hidden**: not by GEMM compute, only partly by the FC weight stream (~55 µs covers 35% of the ~156 µs link time); **every byte matters** (token dedup, relay);
- Two-stage relay shipped through all four stages; combine is its convergent mirror. Decode-only, NVFP4 dispatch, BF16 combine is the validated product path, accepted at world=32, R=8 after a three-hop memory-corruption bug was traced to one wrong zeroing stride (testing matrix missed it out).
- **Tuning against real baselines**: N4R8 3138 → 438 µs, 0.77× NCCL-EP (574.5) and 0.68× DeepEP (653.0).
    - Per-put issue cost (coalescing, -62% -- reproduce before adopting design choices);
    - Fan-out publish: 608 warps × 32 system fences serialize; × one lane-0 fence does not;
    - the FC2 epilogue's per-tile system-scope publication, batch 1×4;
    - the MMA issue chain (pipelined k-loop, -8.3%);
    - Intranode and internode tokens no longer segmented;
    - Design premises fell: Stage 3 removed (relay warp flat assignment -4.2%, direct fan-out -6.4% on top).

Bonus: about vibe coding

- Collect SKILL items along the way
  - All claims must be referenced to file name and line number
  - Mark down the model's implicit design choices



- In AGENTS.md, make MCP queries compulsory (regarding cluster use, internal toolchain PIC-C usage, etc.)
- Change of codepath, change of context
- Use adaptive "effort"

